

ASSIGNMENT # 02

Part - B

SUBMITTED BY

ABDUL REHMAN

SUBMITTED TO

Sir. Umair

Part B - Object-oriented refactor

Take your own To-Do List or ATM project (from your CV) and rewrite it using classes (object-oriented), not just loose functions.

- Example structure: a Task class and a TodoList class - or an Account class for the ATM.
- Add input validation so bad input (wrong type, empty value, out-of-range PIN, etc.) never crashes the program.

Reflection (3–4 sentences): What did organizing your code into classes let you do that loose functions didn't?

```
JS todo.js
Assg-02(B) > JS todo.js > ...
1  class Task {
2      constructor(name) {
3          this.name = name;
4          this.completed = false;
5      }
6      markCompleted() {
7          this.completed = true;
8      }
9  }
10 class TodoList {
11     constructor() {
12         this.tasks = [];
13     }
14     addTask(taskName) {
15         if (!taskName || taskName.trim() === "") {
16             console.log("Task name cannot be empty.");
17             return;
18         }
19         let task = new Task(taskName);
20         this.tasks.push(task);
21         console.log(`Task "${taskName}" added successfully.`);
22     }
23     completeTask(taskName) {
24         let task = this.tasks.find(t => t.name === taskName);
25         if (!task) {
26             console.log("Task not found.");
27             return;
28         }
29         task.markCompleted();
30         console.log(`Task "${taskName}" marked as completed.`);
31     }
32     deleteTask(taskName) {
33         let index = this.tasks.findIndex(t => t.name === taskName);
34         if (index === -1) {
35             console.log("Task not found.");
36             return;
37         }
38         this.tasks.splice(index, 1);
39         console.log(`Task "${taskName}" deleted successfully.`);
40     }
}
```

```

40     }
41     showTasks() {
42         if (this.tasks.length === 0) {
43             console.log("No tasks available.");
44             return;
45         }
46         console.log("\nTo-Do List:");
47         this.tasks.forEach((task, index) => {
48             console.log(
49                 `${index + 1}. ${task.name} - ${task.completed ? "Completed" : "Pending"}`
50             );
51         });
52     }
53 }
54
55
56 // Testing
57
58 let myTodoList = new TodoList();
59
60 myTodoList.addTask("Complete Assignment");
61 myTodoList.addTask("Study JavaScript");
62
63 myTodoList.showTasks();
64
65 myTodoList.completeTask("Complete Assignment");
66
67 myTodoList.showTasks();
68
69 myTodoList.deleteTask("Study JavaScript");
70
71 myTodoList.showTasks();

```

Code Explanation

Class / Method	Code Segment	Description
Task Class	class Task	Represents a single task in the To-Do List. Each task has a name and a completion status.
constructor(name)	this.name = name; this.completed = false;	Initializes a new task by storing its name and setting the completion status to pending (false).
markCompleted()	this.completed = true;	Changes the task status from pending to completed.
TodoList Class	class TodoList	Manages all tasks and provides methods to add, complete, delete, and display tasks.
constructor()	this.tasks = [];	Creates an empty array to store task objects.
addTask(taskName)	if (!taskName taskName.trim() === "")let task = new Task(taskName);this.tasks. push(task);	Validates the task name and adds a new task object to the tasks array.

completeTask(taskName)	let task = this.tasks.find(...);task.markCompleted();	Searches for a task by name and marks it as completed if found.
deleteTask(taskName)	this.tasks.splice(index, 1);	Removes the specified task from the list. If the task is not found, an error message is displayed.
showTasks()	this.tasks.forEach(...);	Iterates through all tasks and displays their current status as Completed or Pending.
Testing	let myTodoList = new TodoList();	Creates a TodoList object and tests all functionalities such as adding, completing, deleting, and displaying tasks.

Output

Output

Clear

```

Task "Complete Assignment" added successfully.
Task "Study JavaScript" added successfully.

To-Do List:
1. Complete Assignment - Pending
2. Study JavaScript - Pending
Task "Complete Assignment" marked as completed.

To-Do List:
1. Complete Assignment - Completed
2. Study JavaScript - Pending
Task "Study JavaScript" deleted successfully.

To-Do List:
1. Complete Assignment - Completed

=== Code Execution Successful ===

```

Reflection:

Using classes made the code more organized by grouping related data and methods together the separation between the Task and TodoList classes improved readability and made the program easier to maintain and extend in the future.